Motore per avventure testuali: documento di design

8 novembre 2026

Visione e obiettivi

Il progetto è un motore per avventure testuali in due parti: un'app web in cui l'autore crea il gioco come insieme di dati (il "core") e dei compilatori che trasformano quel core in un gioco eseguibile su piattaforme moderne e retro. L'avventura si scrive una volta sola e si pubblica ovunque.

Obiettivi:

- Un unico core dei dati, indipendente dalla piattaforma.
- Il parser testuale (l'utente scrive i comandi) come modalità principale di gioco.
- Tre primi target di compilazione: Web/HTML, Commodore 64, ZX Spectrum.
- Un'app di authoring web, usabile anche senza scrivere codice per i casi comuni.
- Più lingue: italiano e inglese fin dall'inizio, una sola lingua per ogni gioco compilato.

Prevista come estensione successiva: una modalità "semplice" a scelte multiple, per chi non vuole digitare i comandi.

Fuori ambito, in questa prima versione (proposta): grafica, audio, multigiocatore.

Utenti e casi d'uso

I due ruoli sono l'autore, che costruisce l'avventura nell'app, e il giocatore, che la gioca sulla piattaforma scelta. Il caso d'uso centrale è: scrivere un'avventura nel browser, provarla subito, poi esportarla per il C64 o lo Spectrum.

Flusso tipico dell'autore:

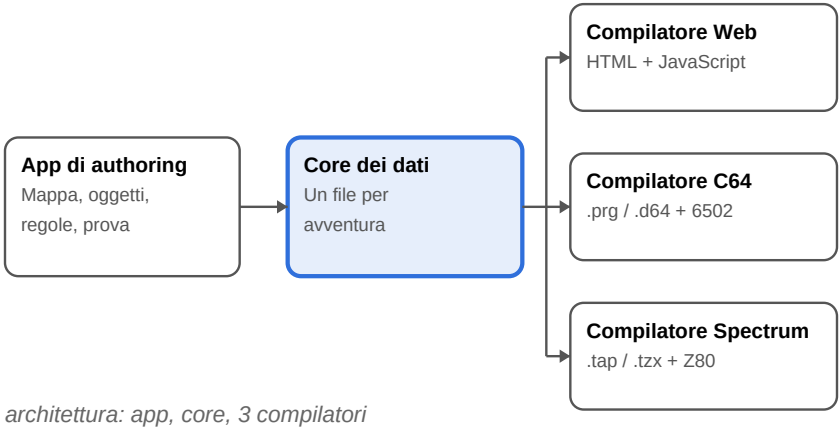
1 Crea un progetto e definisce mappa, oggetti e personaggi.

2. Scrive i testi e le regole (cosa succede quando si usa un oggetto, si apre una porta, ecc.).
3. Prova il gioco dentro l'app, con un terminale di test.
4. Sceglie una piattaforma e compila.
5. Ottiene il file da distribuire: una pagina HTML, un `.tap` o `.tzx` per lo Spectrum, un `.d64` o `.prg` per il C64.

Il formato esatto dei file di uscita è da confermare per ogni backend (vedi la sezione sui target).

Architettura generale

Un solo core alimenta tutti i compilatori



architettura: app, core, 3 compilatori

L'autore lavora solo nell'app e produce il core; ogni compilatore legge quel file e genera il gioco per la sua piattaforma. La schermata Prova dell'app usa lo stesso interprete JavaScript del target Web, così ciò che si vede nel test è ciò che si pubblica.

Il core dei dati

Il core è un solo file di dati (JSON nell'app, poi convertito in forma compatta dai compilatori; il linguaggio leggibile si genera e si importa a richiesta) che descrive tutta l'avventura senza codice di piattaforma. Deve restare abbastanza piccolo e semplice da stare nella memoria di un C64 o di uno Spectrum.

Elemento	Cosa contiene
Stanza	Nome, descrizione, uscite verso altre stanze, eventuali condizioni
Oggetto	Nome, sinonimi, descrizione, stanza o inventario in cui si trova, proprietà (prendibile, contenitore, personaggio, ecc.)
Personaggio	Un oggetto con la proprietà «personaggio»; le risposte sono regole
Flag e contatori	Variabili del mondo di gioco (porta aperta, punti, turni)
Regola	Condizione sul comando e sullo stato, con le azioni che ne derivano
Vocabolario	Verbi, nomi, preposizioni e relativi sinonimi
Testi	Messaggi di sistema e testi del gioco, separati per poterli tradurre
Soluzione	Passaggi di alto livello per finire il gioco (obiettivi su oggetti, luoghi e personaggi), con nota e raggruppati per tappa. Resta nel core ma non viene compilata

Scelte confermate:

- Il core non contiene nulla di specifico di una piattaforma: lunghezza delle righe, codici dei colori e caratteri speciali si gestiscono nel compilatore.
- Ogni elemento ha un identificativo stabile, così i riferimenti non si rompono quando si rinomina qualcosa.
- Il file porta un numero di versione del formato, per poter aggiornare i vecchi progetti.

App di authoring

L'app è una web app che funziona nel browser e salva il progetto come file del core, da scaricare e riaprire. Il salvataggio locale automatico serve da rete di sicurezza, ma il file resta la copia di riferimento.

Schermate principali:

- **Mappa:** stanze come riquadri collegati dalle uscite, da trascinare e collegare.

- **Stanze, oggetti, personaggi:** moduli per nomi, descrizioni, sinonimi e proprietà.
- **Regole:** editor visuale delle condizioni e delle azioni, a moduli con menu, con la vista testuale della stessa regola (nel linguaggio leggibile) su cui si può intervenire direttamente.
- **Vocabolario:** elenco dei verbi e dei sinonimi riconosciuti dal parser.
- **Prova:** terminale che esegue il gioco in tempo reale con lo stesso motore del target Web.
- **Soluzione:** passaggi di alto livello per finire il gioco (prendi un oggetto, usalo altrove, sconfiggi un personaggio), controllati in automatico (vedi sotto).
- **Compila:** scelta della piattaforma, opzioni e download del risultato.

La **Soluzione** serve all'autore per tenere traccia di come si risolve l'avventura, a livello di obiettivi e non di comandi esatti. Ogni passaggio dice cosa succede ai personaggi, agli oggetti e ai luoghi del gioco, più una nota libera, e i passaggi si possono riordinare e raggruppare per tappa. Esempio: "prendi la chiave", "usa la chiave sulla porta della cantina", "sconfiggi il guardiano", "gioco completato".

Tipi di passaggio (proposta):

- **Prendi:** l'eroe ottiene un oggetto.
- **Usa:** un oggetto viene usato in un luogo, su un altro oggetto o su un personaggio.
- **Raggiungi:** l'eroe arriva in un luogo.
- **Parla o attacca:** un'azione verso un personaggio.
- **Ottieni uno stato:** un flag o un contatore assume un valore (la porta è aperta).
- **Fine del gioco:** l'avventura è vinta.

Il controllo automatico funziona così. L'app parte dall'inizio del gioco e usa lo stesso interprete della schermata Prova. Per ogni passaggio cerca una sequenza di comandi che lo realizzi, partendo dalla situazione lasciata dal passaggio precedente: prova i comandi possibili con gli oggetti e le uscite a portata dell'eroe. Se la trova, il passaggio è valido e l'app mostra i comandi trovati, che si possono esportare come soluzione passo passo. Se non la trova, segnala il primo passaggio impossibile e perché (per esempio, manca la chiave). Se l'ultimo passaggio è raggiungibile, il gioco si può finire davvero. Il controllo si rifa dopo ogni modifica alle regole.

Due controlli in più, indipendenti dai passaggi: cercare da soli una strada fino alla vittoria, e segnalare i vicoli ciechi, cioè le situazioni da cui non si può più vincere (per esempio, aver lasciato in un posto irraggiungibile l'oggetto che serve).

Limiti da tenere presenti: nei giochi grandi le combinazioni possibili crescono molto, e i passaggi servono proprio a tenere breve ogni ricerca, perché l'app cerca solo da un passaggio al successivo. Gli eventi casuali o a tempo per ora non fanno parte della soluzione: se ne riparlerà quando verranno implementati. La soluzione è salvata nel core ma non entra nel gioco compilato.

Altri controlli utili dall'inizio: elenco di avvisi (uscite senza destinazione, oggetti mai posizionati, parole usate nei testi ma assenti dal vocabolario) e il calcolo approssimativo di quanta memoria occuperà il gioco su ogni piattaforma retro.

Parser e linguaggio

Il parser deve essere identico in tutti i target, così l'avventura si comporta allo stesso modo ovunque. Per questo è un componente piccolo e ben definito, riscritto per ogni piattaforma (JavaScript per il Web, linguaggio macchina per i retro) a partire da una specifica comune.

Proposta di base, da confermare:

- Comandi nella forma **verbo + nome** e **verbo + nome + preposizione + nome** ("prendi lampada", "apri porta con chiave").
- Ogni lingua ha un parser a sé (vocabolario, articoli, preposizioni, ordine delle parole). La lingua si sceglie al momento della compilazione: il gioco finale contiene un solo parser e i testi di una sola lingua, senza scelta durante il gioco. Si parte da italiano e inglese.
- Sinonimi e abbreviazioni definiti nel vocabolario ("n" per "nord", "i" per "inventario").
- Su sistemi con poca memoria il parser può confrontare solo i primi caratteri di ogni parola (per esempio 4 o 5), come facevano molte avventure d'epoca. La lunghezza è un'opzione del compilatore.
- Messaggi standard per i casi comuni: parola non conosciuta, oggetto non presente, azione impossibile.

La logica delle regole segue lo schema **condizione, azione**: ad esempio, "se il comando è *apri porta*, e il giocatore ha la chiave, allora il flag *porta aperta* diventa vero e compare un messaggio". Le regole sono dati nel core, non codice.

Target di compilazione

Ogni target è un compilatore che legge lo stesso core e produce un gioco autonomo. Il compilatore Web è il più semplice e serve anche da riferimento per il comportamento degli altri.

Target	Uscita prevista	Interprete	Note
Web/HTML	Pagina HTML unica con JavaScript	JavaScript	Funziona anche su mobile; è lo stesso motore della schermata Prova
Commodore 64	File `.prg` o immagine `.d64`	Linguaggio macchina 6502	Testi compressi e caricati a blocchi
ZX Spectrum	File `.tap` o `.tzx`	Linguaggio macchina Z80	Testi compressi, schermo a 32 colonne

Il compilatore per un sistema retro lavora in tre passi: converte il core in tabelle compatte (stanze, oggetti, regole, vocabolario), comprime i testi, e li unisce a un interprete già pronto per quella macchina, producendo il file finale. L'interprete è scritto una volta sola per piattaforma e non cambia da un'avventura all'altra; cambiano solo i dati.

Ogni interprete retro esiste in una versione per lingua, già assemblata in linguaggio macchina. Il compilatore quindi non assembla nulla: sceglie il binario della lingua giusta e ci aggiunge i dati. Per questo la compilazione avviene nel browser, dentro l'app, se tecnicamente possibile.

Vincoli dei sistemi retro

I target retro impongono limiti che il core e l'app devono tenere presenti fin dall'inizio, altrimenti si scoprono solo alla compilazione.

Vincolo	C64	ZX Spectrum
Memoria	64 KB di RAM, in parte occupata dal sistema	48 KB di RAM nel modello base (esiste anche il 16 KB)
Larghezza testo	40 colonne	32 colonne
Set di caratteri	Non contiene tutte le lettere accentate italiane	Idem, con set modificabile

Conseguenze sul design:

- **Budget di memoria:** l'app mostra quanto spazio occupano testi, regole e vocabolario per ogni target, e avvisa prima di superare il limite.
- **Compressione dei testi:** i testi sono la parte più grande di un'avventura, quindi il compilatore li comprime (per esempio con dizionari di parole o di sillabe frequenti).
- **Caratteri speciali:** all'inizio le lettere accentate si scrivono come vocale più apostrofo (e', a', o'); una soluzione migliore, come ridefinire i caratteri nel set grafico, si valuta più avanti.

- **Disposizione del testo:** il compilatore va a capo da solo in base alla larghezza dello schermo, quindi l'autore non inserisce a capo manuali.
- **Salvataggio della partita:** da decidere per ogni piattaforma (su nastro o disco per i retro, nel browser per il Web).
- **Avventure grandi:** se non entrano in memoria, valutare il caricamento a blocchi da disco (C64) o a più parti (Spectrum).

Roadmap

Lo sviluppo procede per fasi, ognuna con un risultato giocabile, così ogni passo si può provare prima di aggiungere il successivo.

1. **Core e parser Web:** definire il formato del core e scrivere l'interprete JavaScript, con i parser italiano e inglese. Risultato: un'avventura di prova scritta a mano come file di dati, giocabile nel browser.
2. **App di authoring minima:** moduli per stanze, oggetti e vocabolario, più la schermata Prova. Risultato: creare e provare un'avventura senza toccare i file a mano.
3. **Editor delle regole e mappa:** regole condizione/azione e mappa visuale, con gli avvisi di controllo, la schermata Soluzione e il controllo automatico dei passaggi.
4. **Primo target retro:** compilatore e interprete per il primo sistema (scelta tra C64 e Spectrum), con compressione dei testi e budget di memoria.
5. **Secondo target retro:** adattare il compilatore all'altro sistema, riusando le tabelle e la specifica del parser.
6. **Rifiniture:** salvataggio partite, esportazione e importazione dei progetti (JSON e linguaggio leggibile), esempi e documentazione. Poi, come estensione, la modalità a scelte multiple.

Metodo: si procede per passi piccoli, con poche funzioni nuove alla volta, da provare prima di andare avanti. Prototipo giocabile

nel browser, con il core in JSON modificabile nella pagina di prova. Passo 1 (fatto): stanze, uscite e oggetti, con guarda, vai, prendi, lascia e inventario. Passo 2 (fatto): una porta chiusa a chiave che si apre con "apri porta con chiave", il primo comando con due oggetti. Passo 3 (fatto): oggetti usabili con "usa", ad esempio una lanterna che si accende e si spegne. Passo 4 (fatto): un'uscita nascosta in cantina che compare solo con la lanterna accesa. Finora queste funzioni sono fisse nel motore; nell'app finale diventeranno regole con condizione e azione, da scrivere con l'editor visuale. Poi è nata l'app di authoring, con schede per stanze, oggetti, parole e testi, prova del gioco e core JSON. Le stanze possono avere frammenti di descrizione condizionati dallo stato degli oggetti, le uscite di ritorno si creano da sole e un oggetto può restare fuori dall'elenco «Qui vedi» perché nominato nella descrizione. Ultimo passo (fatto): le regole, cioè «quando il giocatore scrive questo comando, se valgono queste condizioni, succedono queste azioni». Le condizioni sono lo stato di un oggetto (aperto, chiuso, acceso, spento, qui, con te) o la stanza in cui ci si trova. Le azioni sono dire un testo, spostare un oggetto, aprire, chiudere, accendere, spegnere e terminare il gioco. Vale la prima regola che combacia; se nessuna agisce, il comando fa quello che farebbe di base. Poi le funzioni fisse sono state riscritte come regole: porte, uso degli oggetti, finale e uscite nascoste ora sono dati, e il motore di base conosce solo cinque comandi (guarda, vai, prendi, lascia, inventario). Le uscite possono avere condizioni, un testo se sono bloccate e restare nascoste finché non diventano disponibili; le condizioni accettano anche «qui o con te». Il formato del core è ora la versione 2. Infine la schermata Soluzione: i passi (prendi un oggetto, raggiungi una stanza, porta un oggetto in una stanza, ottieni uno stato, fine del gioco) si verificano esplorando tutte le partite possibili con lo stesso motore del gioco. La verifica dice se i passi si raggiungono nell'ordine, mostra il percorso più corto e trova i vicoli ciechi, cioè le situazioni da cui non si può più finire il gioco. Sull'esempio, 11.694 situazioni si esplorano in pochi secondi; per giochi molto grandi l'esplorazione si interrompe e lo dice. La verifica ha scovato un vero vicolo cieco nell'esempio (lasciare la lanterna nel passaggio segreto), poi corretto con una regola. Poi il primo

compilatore, quello Web: dal core produce un solo file HTML con dentro il motore e i dati, che si apre in qualsiasi browser anche senza connessione, senza nessuna risorsa esterna. È la stessa libreria del gioco usato per la prova, quindi il gioco compilato si comporta come in anteprima; la soluzione non viene inclusa. L'esempio compilato pesa circa 21 KB. Nell'app, la scheda Compila offre anteprima, salvataggio del file e copia del codice. Poi il compilatore per Commodore 64, a passi piccoli. Il programma finale è un file PRG fatto di tre parti: un breve avvio BASIC (SYS 2061), l'interprete già scritto in codice macchina 6502 e i dati del gioco, che il compilatore costruisce nel browser a un indirizzo fisso. I testi sono convertiti in PETSCII, con le lettere accentate scritte come vocale più apostrofo, e a capo prima della colonna 40. Primo passo (fatto): stanze, uscite e i comandi «guarda» e «vai»; oggetti, regole e uscite con condizioni non sono ancora inclusi e il compilatore lo segnala. L'esempio pesa circa 2 KB. Il test è stato fatto con un emulatore 6502 scritto per l'occasione, non ancora con un vero C64 o con VICE. L'app salva un file zip contenente il PRG. Passo 2 (fatto): oggetti con prendi, lascia, inventario e guarda un oggetto. Passo 3 (fatto): le regole in codice macchina, cioè comandi con uno o due oggetti, condizioni (stato di un oggetto, stanza), azioni (dire, spostare, aprire, chiudere, accendere, spegnere, descrivere, fine), testi alternativi, stati che cambiano etichette e descrizioni, uscite con condizioni, bloccate o nascoste con rivelazione. L'esempio completo, dalla chiave alla moneta, si porta a termine nell'emulatore e pesa circa 5 KB. Passo 4 (fatto): l'interprete va a capo da solo tra le parole sullo schermo a 40 colonne, quindi i testi non sono più preparati in righe dal compilatore, e i frammenti di descrizione condizionati sono inclusi. Ora il compilatore C64 copre tutto il formato del core; il test su VICE ha confermato che funziona. Poi lo ZX Spectrum, con lo stesso metodo: assemblatore ed emulatore Z80 scritti per l'occasione, interprete in codice macchina Z80, caricato all'indirizzo $8000, dati con la stessa struttura del C64 (la parte del compilatore che costruisce i dati è condivisa), testi in ASCII con le vocali accentate scritte come vocale più apostrofo, tastiera letta direttamente dalla matrice e a capo automatico su 32 colonne. Il file prodotto è un .tap con un breve caricatore BASIC. Primo

passo (fatto): stanze, uscite, «guarda» e «vai», che sul vero Spectrum funziona. La tastiera riconosce ogni tasto alla pressione, anche con un altro tasto ancora premuto, e ripete i tasti tenuti premuti (utile per cancellare molti caratteri). Passo 2 (fatto): oggetti con prendi, lascia, inventario e guarda un oggetto, provato nell'emulatore Z80. Passo 3 (fatto): regole, stati degli oggetti, uscite con condizioni e frammenti di descrizione, con lo stesso formato dei dati del C64. L'esempio si porta a termine nell'emulatore Z80. Ora i compilatori Web, C64 e Spectrum coprono tutto il formato del core. In tutti i giochi «g» è un'abbreviazione di «guarda». Nell'app, la scheda Core permette di salvare il core in un file e di riaprirlo (con una conferma prima di sostituire il lavoro in corso e il controllo che il file sia valido). Infine la scheda Mappa: mostra in sola lettura le stanze come scatole disposte seguendo le direzioni a partire dalla stanza iniziale, con linee per le uscite (tratteggiate se hanno condizioni, bloccate o nascoste), frecce dove si passa in una sola direzione e un livello separato per le uscite su e giù. Segnala le stanze che non si raggiungono dall'inizio e quelle disegnate in posizione approssimativa perché le direzioni non tornano; toccando una stanza si apre la sua scheda. Possibili sviluppi, non ancora fatti: trascinare le stanze con posizioni salvate nel core, oggetti e percorso della soluzione sulla mappa. Poi la leggibilità dei giochi retro: ogni testo ha un tipo (nome della stanza, testo, oggetti, uscite, errori, comando del giocatore), segnato nei dati con un byte da 1 a 6 che non occupa spazio sullo schermo, e ogni interprete sceglie il colore del tipo. Sfondo nero su entrambi. C64: nome della stanza giallo, testo grigio chiaro, oggetti verde chiaro, uscite ciano, errori rosso chiaro, comando bianco. Spectrum (con BRIGHT): nome giallo, testo bianco, oggetti verde, uscite ciano, errori rosso, comando bianco brillante. In seguito i colori potranno stare nel core, modificabili dall'autore. Sul C64 il gioco usa anche un font proprio, disegnato per l'occasione (licenza MIT): tratti sottili, lettere larghe 6 punti, maiuscole alte 6 righe, minuscole con aste e gambe, una riga vuota sopra e sotto per separare le righe di testo. Il compilatore mette il font a $1400 e i dati del gioco a $1700; all'avvio l'interprete copia il font sotto il KERNAL a $F000 e sposta lo schermo a $C800, nel banco 3 del VIC, così non

toglie memoria ai dati. Poi la leggibilità dell'app di authoring, primo passo: un colore per tipo di elemento (stanze blu, oggetti verde, regole arancio), ripreso nelle schede in alto, nel bordo delle card e nei riferimenti, che appaiono come etichette colorate (per esempio l'oggetto di una regola o la stanza di un oggetto). Le card chiuse mostrano un riassunto (uscite e frammenti di una stanza, condizioni e azioni di una regola). Dentro le card i titoli di sezione sono evidenti e i campi da compilare hanno un bordo netto. Le regole sono divise in quattro blocchi colorati: Quando, Solo se, Allora, Altrimenti. Sul telefono le schede in alto stanno su tre righe, tutte visibili. Nella Prova e nel gioco Web compilato i testi hanno gli stessi tipi dei giochi retro: nome della stanza, oggetti, uscite ed errori con colori diversi, e il comando del giocatore in grassetto con un segno a sinistra. Secondo passo: tema scuro con sfondi più scuri e un selettore Auto / Chiaro / Scuro in alto a destra (la scelta resta nel browser). Le schede che mancavano hanno lo stesso trattamento: Parole con un suo colore (viola), i comandi e le direzioni come etichette; Soluzione con i passi numerati e riassunti con etichette di oggetti e stanze, esito della verifica in riquadri verdi o rossi; Compila con un colore per piattaforma e la dimensione in evidenza; Core con «Salva e apri» in cima e il JSON sotto. Poi i testi nascosti: nel gioco Web il core (senza soluzione) è compresso con lo strumento del browser, rimescolato con una chiave casuale e scritto in base64, e il gioco lo decodifica all'avvio; nel sorgente della pagina non si legge nulla dell'avventura e il file è più piccolo (l'esempio passa da 21 a 19 KB). Su C64 e Spectrum ogni testo è rimescolato byte per byte (XOR con una chiave che cresce a ogni carattere) e le lettere del vocabolario con una chiave fissa, così non si leggono con un editor esadecimale o un monitor; il costo è di poche decine di byte di codice. Non è una protezione vera, perché il gioco deve poter leggere i suoi testi, ma impedisce di sbirciare per caso. La compressione dei testi sui sistemi retro, che farebbe anche risparmiare memoria, resta per più avanti. Poi le variabili e gli eventi d'ingresso (primi passi verso la prima versione): vedi i capitoli dedicati.

Salvataggi e dati grandi sui sistemi retro (proposta)

Per ora non c'è nessun salvataggio: questa è la proposta da cui partire quando servirà.

Cosa si salva. Lo stato della partita è piccolo: stanza attuale, posizione e stato di ogni oggetto, fine del gioco; con 100 oggetti sono circa 200 byte. Conviene tenerlo in un unico blocco di memoria contiguo, così salvare significa scrivere un solo blocco. Il blocco ha una piccola intestazione: una firma, un codice del gioco (calcolato dal compilatore sul contenuto) e una somma di controllo, per rifiutare salvataggi di un altro gioco, di un'altra versione o rovinati.

Comandi. «salva» e «carica» diventano comandi di base, con le parole modificabili nella scheda Parole. In più un salvataggio in memoria («salva rapido» / «ricarica»), come nelle avventure dell'epoca: copia lo stato in un'area libera della RAM, è istantaneo e non richiede nessun dispositivo; utile prima di una mossa rischiosa.

Commodore 64, disco. Si usano le routine del KERNAL (SETLFS, SETNAM, SAVE e LOAD) sull'unità 8: il file contiene solo il blocco di stato. Per sovrascrivere un salvataggio si cancella prima il file con il comando S: sul canale 15, invece di usare «@0:», che sul 1541 ha un difetto noto. Gli errori del drive si leggono dal canale 15 e si mostrano come messaggio di errore del gioco. Il nome lo sceglie il giocatore (massimo 16 caratteri) oppure si usano posti numerati (1-9).

Commodore 64, nastro. Stesse routine con l'unità 1: il KERNAL mostra da solo i messaggi «PRESS RECORD & PLAY». Lento ma funziona su qualsiasi C64; il gioco può chiedere all'inizio dove salvare (disco o nastro).

Commodore 64, cartuccia. La ROM della cartuccia non si scrive, quindi anche i giochi su cartuccia salvano su disco o nastro, se il KERNAL resta visibile (le cartucce tipo Magic Desk ed EasyFlash lo permettono). Sulla EasyFlash si potrebbe anche scrivere nella memoria flash della cartuccia (con le routine EAPI), ma si cancella a settori interi da 64 KB e si consuma: lo terrei come opzione futura.

Commodore 64, avventure grandi. La parte del gioco che serve sempre resta in RAM: interprete, vocabolario, tabelle di stanze, uscite, oggetti e regole, stato. I testi, che sono la parte più grande, possono stare fuori e venire letti quando servono. L'interprete ha una sola routine «prendi il testo numero N», e cambia solo il modo in cui la routine lo trova:

- **In RAM** (come oggi), per i giochi che ci stanno.
- **Da disco:** il compilatore produce direttamente un'immagine .d64 con il programma e i blocchi di testo in settori noti; l'interprete legge il settore con il comando U1 del drive, senza cercare file nella directory, e tiene in una cache in RAM gli ultimi blocchi usati. Raggruppando i testi per zona del gioco, muovendosi nella stessa zona non si carica quasi nulla. Funziona su 1541 reale, SD2IEC e Ultimate. Niente fast loader all'inizio, per compatibilità; eventualmente come opzione.
- **Da cartuccia:** i testi stanno nei banchi della ROM (Magic Desk/Ocean fino a 512 KB, EasyFlash fino a 1 MB), visibili a finestre di 8 KB a $8000; l'interprete seleziona il banco, copia il testo in un buffer e lo stampa. Nessuna attesa. Serve una variante dell'interprete che parte dalla cartuccia e si copia in RAM.

ZX Spectrum, nastro. Le routine della ROM per salvare e caricare un blocco con intestazione (SA-BYTES a $04C2, LD-BYTES a $0556) vanno bene per il blocco di stato: pochi secondi. Il nome del file ha al massimo 10 caratteri. Funziona su tutti i modelli.

ZX Spectrum 128K, +2, +3. Hanno 128 KB divisi in banchi da 16 KB, selezionati dalla porta $7FFD: un'avventura grande si carica tutta all'avvio nei banchi liberi e poi non serve più leggere nulla durante il gioco. Uno dei banchi può anche ospitare il salvataggio in memoria.

ZX Spectrum +3, disco. Il +3DOS (nella ROM 2) offre apertura, lettura e scrittura di file: si usa sia per i salvataggi sia per leggere i testi a blocchi, come sul C64.

ZX Spectrum con DivMMC o DivIDE (esxDOS). Oggi è il modo più diffuso tra gli appassionati: con le chiamate RST $08 si aprono, leggono, scrivono e posizionano file sulla scheda SD. Salvataggi istantanei e lettura casuale dei testi, quindi avventure grandi anche su un 48K. L'Interface 1 con i Microdrive è possibile ma poco diffusa: la lascerei fuori.

ZX Spectrum 48K su nastro, avventure grandi. Il nastro non permette di leggere un testo a caso. Si fa come le avventure dell'epoca: il gioco è diviso in parti; alla fine di una parte il gioco salva lo stato e chiede di caricare la parte successiva, che riparte da quello stato.

Riepilogo. Lo stesso interprete con tre scelte fatte dal compilatore secondo il supporto scelto dall'autore: dove stanno i testi (RAM, disco, cartuccia, banchi 128K, file esxDOS, parti su nastro), dove si salva (disco, nastro, SD, memoria) e quindi quale variante dell'interprete includere. I salvataggi in memoria e lo stato in un blocco unico conviene prepararli per primi, perché servono a tutte le varianti. Gli emulatori offrono comunque le istantanee, che per chi gioca in emulazione sostituiscono il salvataggio.

Verso una prima versione utilizzabile (proposta)

Criterio: un autore deve poter scrivere un'avventura vera, piccola (15-30 stanze), senza scontrarsi con i limiti del motore, e giocarla sulle tre piattaforme. Le funzioni sono raggruppate per ambito; l'ordine di lavoro è in fondo. Il salvataggio partita resta fuori, per scelta.

Il motore: cose che un'avventura vera incontra subito

- **Variabili (flag e contatori) — fatto:** condizione «se X vale / è almeno N» e azione «imposta / aumenta X». Servono per i dialoghi con i personaggi, gli interruttori, il «l'hai già fatto» e il punteggio. Su C64 e Spectrum sono un piccolo array di byte, con poco codice in più.

- **Eventi d'ingresso — fatto:** regole che scattano quando il giocatore entra in una stanza (sempre, o solo la prima volta, usando una variabile). Per scene, trappole, incontri. Gli eventi a tempo e casuali restano per dopo.
- **Aggettivi — fatto:** «chiave d'oro» e «chiave arrugginita», «porta nord»; se la parola è ambigua il gioco chiede «Quale?». Oggi il vocabolario ha una parola per oggetto, e due oggetti con lo stesso nome si incontrano presto.
- **Contenitori — fatto:** «apri scatola» che mostra il contenuto, «metti X in Y», come proprietà dell'oggetto.
- **Personaggi e argomenti — fatto:** oggetti con la proprietà «personaggio», elenco «Qui c'è…», «dai X a Y», argomenti di conversazione e modelli di regola. Restano i personaggi che si muovono e i dialoghi a menu.
- **Stanze buie — fatto:** una stanza può essere «buia» e si vede solo con una fonte di luce accesa, un classico.
- **Comandi comuni — fatto:** «aspetta» (z), «ripeti» (r), «aiuto», «prendi tutto» e «lascia tutto».
- **Lingua inglese** (decisione già presa): vocabolario di base e messaggi predefiniti in inglese, un parser per lingua scelto in compilazione.
- **Lettere accentate vere sulle versioni retro:** sul C64 il font è nostro, quindi aggiungere à è é ì ò ù costa pochi byte; sullo Spectrum con i caratteri definibili (UDG). Sostituisce il «vocale più apostrofo».

L'app di authoring

- **Nuovo progetto e più progetti:** elenco, rinomina, duplica; oggi c'è un solo core nel browser. Con **Annulla**, perché sul telefono si sbaglia facilmente un tocco.
- **Modelli di regola:** «porta chiusa a chiave», «interruttore», «contenitore», «personaggio che dà un oggetto in cambio di un altro»: riempiono la scheda Regole con regole pronte da ritoccare.

- **Pannello Controlli:** stanze irraggiungibili, oggetti che non si trovano mai, regole che non scattano mai, parole duplicate, testi troppo lunghi o con caratteri non supportati, memoria usata per piattaforma. Oggi i controlli sono sparsi tra Mappa, Soluzione e Compila.
- **Anteprima «look retro»** nella Prova: 40 o 32 colonne e colori, per vedere come andrà a capo prima di compilare.
- **Dati dell'opera:** autore, versione, crediti; schermata del titolo; comando «crediti».

Uscite e rifiniture

- C64: file **.d64** oltre al .prg, **più avanti**: arriverà insieme al caricamento dei testi da disco, per le avventure grandi. Spectrum: schermata di caricamento opzionale.
- Un **esempio più ricco** di quello attuale (10 stanze o più, che usi tutte le funzioni). La **guida dell'app — fatta** (versione 49): un documento a parte, «Guida all'app per avventure testuali», che spiega scheda per scheda come si scrive un'avventura. Con la guida l'app ha anche il riquadro «Titolo e introduzione» nella scheda Stanze, e tutte le frasi di risposta sono modificabili nella scheda Parole.
- Prova sul **hardware reale e sul televisore CRT**, pulizia delle licenze, README e scelta di dove ospitare l'app.

Ordine proposto. Prima le variabili e gli eventi d'ingresso, perché sbloccano dialoghi e scene; poi Annulla e i progetti multipli, che proteggono il lavoro dell'autore; poi gli aggettivi e le lettere accentate vere; poi l'inglese, il pannello Controlli e i modelli di regola; per ultimi contenitori, stanze buie e comandi comuni, che con le regole si possono già aggirare. Il minimo per chiamarla versione 1: variabili, eventi d'ingresso, aggettivi, inglese, accenti, progetti con Annulla, Controlli, .d64, esempio e guida.

Variabili (fatto)

Una variabile è un numero da 0 a 255 che il gioco ricorda: i tentativi fatti, i punti, quante volte hai parlato con un

personaggio. Nel core sta in un elenco facoltativo «variabili», ognuna con un identificativo, un nome, un valore iniziale e un valore massimo (di base 255). Il formato del core resta 2: un core senza variabili si comporta come prima, e le vecchie versioni dei core si aprono senza modifiche.

- **Condizioni:** «la variabile è uguale / diversa / almeno / al massimo N», da usare come tutte le altre condizioni di una regola, di un'uscita o di un frammento di descrizione.
- **Azioni:** «imposta» (il valore si ferma al massimo), «aumenta di N» (si ferma al massimo) e «diminuisci di N» (si ferma a zero). Non ci sono mai valori fuori scala, quindi nessun errore a metà partita.
- **Soluzione:** nuovo tipo di passo «una variabile arriva a un valore», verificato con lo stesso motore del gioco.
- **Controlli:** al massimo 64 variabili; il core non è valido se un valore iniziale supera il massimo o se una regola cita una variabile che non esiste; una variabile non si può eliminare finché qualcosa la usa.

Attenzione al costo della verifica. La verifica della soluzione esplora tutte le partite possibili, e il valore di ogni variabile fa parte della situazione: un contatore da 0 a 255 moltiplicherebbe le situazioni per 256. L'app suggerisce perciò un massimo piccolo (nell'esempio, i tentativi alla porta vanno da 0 a 2 e le situazioni passano da circa 11.700 a circa 28.500, esplorate lo stesso per intero e senza vicoli ciechi).

Nei giochi. Web: le variabili sono parte dello stato della partita. Retro: un piccolo array di byte in RAM (a $C180 sul C64, a $F300 sullo Spectrum), riempito all'avvio da una tabella dei valori iniziali; l'intestazione dei dati passa da 24 a 28 byte. Le condizioni sul tipo «variabile» occupano gli stessi 3 byte delle altre (tipo, indice, valore) e le tre nuove azioni sono nel flusso delle regole; il valore massimo viene deciso dal compilatore, così l'interprete aggiunge poche decine di byte di codice (C64: da $080D a $121A; Spectrum: fino a $8A54). L'esempio ora ha una porta che, al terzo tentativo a vuoto, dà un suggerimento; il comportamento è identico su Web, C64 e Spectrum (verificato

con una prova di tutti gli operatori e dei limiti dei valori). Nella scheda Prova una riga mostra i valori delle variabili durante la partita.

Eventi d'ingresso (fatto)

Un evento d'ingresso è una regola che scatta da sola quando il giocatore entra in una stanza, subito dopo la descrizione. Nel core sta in un elenco facoltativo «eventi» (il formato resta 2): ogni evento ha una stanza (o nessuna, e vale per ogni stanza), un'opzione «solo la prima volta», condizioni facoltative e azioni, le stesse delle regole. Scattano tutti gli eventi che combaciano, nell'ordine in cui sono scritti. Il gioco li esegue anche all'inizio, nella stanza di partenza. Nell'app stanno in fondo alla scheda Regole, sotto le regole.

- **Solo la prima volta:** nel gioco Web il ricordo sta nello stato della partita; sui sistemi retro il compilatore aggiunge una variabile nascosta (da 0 a 1) per ogni evento di questo tipo e la controlla e la imposta da solo. Le variabili nascoste contano nel limite di 64.
- **Retro:** l'evento è una regola con un comando speciale ($FE) e la stanza al posto dell'oggetto; l'interprete la esegue con la stessa routine delle regole, in una modalità che non si ferma alla prima che agisce. Il costo è di circa 80 byte di codice in più.
- **Soluzione:** la verifica parte già dopo gli eventi d'inizio e li applica a ogni spostamento, quindi i passi che dipendono da un evento (per esempio una variabile che cresce entrando in una stanza) si verificano come tutto il resto.
- **Esempio:** entrando la prima volta in cantina si sente un'umidità gelida. Nell'esempio le situazioni da esplorare salgono a circa 28.800.

Provato con eventi all'inizio, in ogni stanza, solo la prima volta, con condizioni su variabili e oggetti: Web, C64 e Spectrum danno gli stessi risultati. Eventi a tempo o casuali, e l'uscita da una stanza come evento, restano per dopo.

Annulla e progetti multipli (fatto)

Annulla e Ripristina. Due pulsanti in alto (■ e ■), e Ctrl+Z / Ctrl+Maiusc+Z su computer. L'app conserva fino a 100 istantanee del core. Le modifiche ravvicinate (meno di un secondo l'una dall'altra, per esempio mentre si scrive) contano come una sola, quindi un tocco toglie una frase intera. Valgono anche per le eliminazioni, l'importazione di un core e «Ripristina l'esempio». La cronologia vive finché l'app è aperta; il core, invece, si salva sempre.

Progetti multipli. Nella scheda Core, in cima, c'è l'elenco dei progetti salvati nel browser: ogni avventura è un core a sé, con il titolo del gioco come nome. Si può aprire un altro progetto, crearne uno vuoto, crearne uno dall'esempio, duplicare quello aperto (utile per provare una variante senza rischiare l'originale) ed eliminarne uno, con doppio tocco di conferma. Non si può eliminare l'ultimo rimasto. Ogni progetto ha la sua cronologia di Annulla finché l'app resta aperta. Il core salvato dalle versioni precedenti diventa il primo progetto, senza perdere nulla.

Limiti noti. I progetti stanno nel browser di questo dispositivo: per averli altrove o per un backup si usa «Salva in un file (.json)», progetto per progetto. Un salvataggio di tutti i progetti in un solo file potrebbe venire in seguito.

Aggettivi (fatto)

Gli aggettivi servono a distinguere oggetti che si chiamano allo stesso modo: «chiave arrugginita» e «chiave dorata». Nel core ogni oggetto può avere un elenco facoltativo «aggettivi» (singole parole); il formato resta 2 e i core senza aggettivi funzionano come prima. Nell'app il campo sta nella scheda dell'oggetto, sotto «Altri nomi».

- **Come si legge un comando.** Per ogni oggetto di cui si parla (prima e dopo la preposizione) il gioco cerca tra gli oggetti presenti, in stanza e con te, quelli che hanno almeno uno dei nomi scritti e tutti gli aggettivi scritti. L'ordine delle parole non conta: «chiave dorata», «dorata chiave» e perfino solo «dorata» vanno bene.

- **Ambiguità.** Se più oggetti rispondono allo stesso nome e gli aggettivi non bastano, il gioco chiede «Quale intendi? Sii più preciso, per esempio con un aggettivo.» (frase modificabile, come le altre). Nelle regole l'ambiguità vale «nessun oggetto», così non scatta una regola per sbaglio.
- **Controlli.** Gli aggettivi devono essere una sola parola. L'app avvisa se due oggetti hanno lo stesso nome e nessun aggettivo li distingue.
- **Esempio.** In cantina c'è una chiave dorata appesa a un gancio (non si può prendere). Se lasci la chiave arrugginita in cantina, «guarda chiave» chiede quale, mentre «guarda dorata» o «guarda chiave arrugginita» funzionano.

Sui sistemi retro. Ogni nome e ogni aggettivo diverso ha un numero (al massimo 254 per tipo); il vocabolario ha un nuovo tipo di parola, l'aggettivo, e le preposizioni sono distinte dalle parole ignorate. Ogni oggetto ha l'elenco dei suoi numeri. Il comando viene diviso in due parti alla prima preposizione e l'interprete cerca gli oggetti con la stessa regola del Web. Lo stesso comando, provato su 29 casi (ambiguità, aggettivi in ogni posizione, aggettivi soli, oggetti in mano o per terra), dà gli stessi risultati su Web, C64 e Spectrum. L'interprete C64 cresce di circa 500 byte, quindi il font ora sta a $1600 e i dati del gioco a $1900. Sullo Spectrum la riga digitata è lunga al massimo 30 caratteri: un comando con aggettivi lunghi può non starci.

Contenitori (fatto)

Un oggetto può essere un **contenitore**: una scatola, un baule, una cassetta. Gli altri oggetti possono stare dentro fin dall'inizio («Dove si trova all'inizio: Dentro: cassetta») o finirci durante il gioco. Nell'esempio, la lanterna ora è dentro la cassetta degli attrezzi del capanno, che va aperta.

- **Nel core.** Due proprietà facoltative dell'oggetto, «contenitore» e «chiudibile»; la posizione di un oggetto può essere l'identificativo di un contenitore. Il formato resta 2. Il controllo del core verifica che la posizione sia un contenitore, che nessun oggetto stia dentro sé stesso e che i contenitori non formino un giro.

- **Chiudibile.** Se un contenitore si apre e si chiude, quello che c'è dentro si vede e si prende solo quando è aperto. Si apre con le regole (azione «Apri»): il modello «Contenitore che si apre e si chiude» le crea, e può aggiungere il comando «chiudi» se manca. Quando una regola apre un contenitore, il gioco dice da solo che cosa c'è dentro («Dentro c'è: una lanterna.»).
- **Nel gioco.** «guarda cassetta» mostra anche il contenuto (o «Dentro non c'è niente.»); «prendi lanterna» la prende anche da un contenitore aperto, nella stanza o che hai con te; «metti chiave nella cassetta» funziona se hai la chiave e la cassetta è presente e aperta. Le regole possono intercettare «metti» per gli enigmi («metti moneta nella fessura»).
- **Comando «metti».** È un comando del motore come prendi e lascia, ma facoltativo: vale se è nel vocabolario. I core nuovi e l'esempio lo hanno, insieme alle preposizioni «in, nel, nella, nello, dentro, da, dal, dalla, dallo» (prima «in», «nel» e «nella» erano parole ignorate). Per i core vecchi la scheda dell'oggetto propone «+ Aggiungile».
- **Posizione effettiva.** Un oggetto dentro un contenitore conta come se fosse dove sta il contenitore (nella stanza o con te), purché i contenitori siano aperti. Vale per le condizioni («è qui», «ce l'hai con te»), per i passi della soluzione e per trovare gli oggetti nei comandi. L'elenco «Qui vedi» mostra solo quello che sta direttamente nella stanza.
- **Sistemi retro.** Una posizione $80+k vuol dire «dentro l'oggetto k»; due bit nuovi nei flag dell'oggetto; il comando «metti» ha l'id 6, quindi i comandi aggiunti dall'autore partono da 7; sette messaggi nuovi. Sul C64 il codice è cresciuto: il font ora sta a $1800 e i dati a $1C00.
- **Controlli.** Un oggetto dentro un contenitore che nessuna regola apre viene segnalato; un oggetto in un contenitore si considera nella stanza del contenitore.

Limiti. Non c'è una capienza (un contenitore accetta qualsiasi numero di oggetti) né il «sopra» (un tavolo con oggetti appoggiati): si possono aggiungere più avanti. I comandi dei

giocatori sullo Spectrum restano limitati a circa 28 caratteri, quindi «prendi lanterna dalla cassetta» non ci sta: basta «prendi lanterna».

Personaggi e argomenti (fatto)

I personaggi sono **oggetti con la proprietà «personaggio»**, non una categoria nuova: hanno già posizione, nomi, aggettivi, descrizioni, condizioni, traduzioni e sono già compilati sui sistemi retro. Il loro comportamento lo decidono le regole, aiutate da due modelli. Nell'esempio, in giardino c'è un vecchio giardiniere con cui parlare e a cui dare la mela.

- **Nel core.** Una proprietà facoltativa dell'oggetto, «personaggio»: il formato resta 2. Il controllo del core verifica che un personaggio non sia prendibile e non stia con il giocatore.
- **Elenco a parte.** Un personaggio non compare in «Qui vedi», ma in una riga sua: «Qui c'è un vecchio giardiniere.», oppure «Qui ci sono Anna e un cane.» se sono più di uno. Come per gli altri oggetti, l'etichetta può cambiare e l'oggetto può essere tolto dall'elenco.
- **«prendi anna».** Risponde «Non sembra d'accordo.» (frase modificabile, tradotta come le altre) e non prende niente. Una regola può dare una risposta più adatta al personaggio.
- **«dai X a Y».** In italiano «a», «ad», «al», «alla» e «allo» sono ora parole di collegamento (prima erano ignorate); in inglese «to» lo era già. Nella scheda del personaggio, se mancano nel vocabolario, c'è «+ Aggiungile».
- **Modello «Parlare con un personaggio».** Il personaggio risponde in modo diverso la prima volta, la seconda e dalla terza in poi. Crea le regole, una variabile che conta le volte e, se manca, il comando «parla». Se si lascia vuota la seconda risposta, le risposte sono due.

- **Modello «Dare un oggetto a un personaggio».** «dai mela al giardiniere»: l'oggetto passa al personaggio e sparisce dal gioco; facoltativamente il personaggio dà in cambio un altro oggetto, che finisce in mano al giocatore. Se il giocatore non ha l'oggetto, risponde con una frase sua.
- **Controlli.** Un personaggio a cui nessuna regola risponde viene segnalato, come una regola che dà qualcosa a un personaggio quando manca la parola «a» tra quelle di collegamento.
- **Sistemi retro.** Un bit nuovo nei flag dell'oggetto (32 = personaggio) e cinque messaggi nuovi (35-39). Gli interpreti C64 e Spectrum elencano gli oggetti in due passate, prima gli altri e poi i personaggi, e scelgono singolare o plurale secondo il numero. Su Web, C64 e Spectrum le risposte del giardiniere coincidono in italiano e in inglese.

Argomenti di conversazione. Un argomento è un oggetto con la proprietà «argomento»: non sta in nessuna stanza («dove» vale «nessuno»), non si vede né si guarda né si prende, ma le regole lo riconoscono anche se non è presente. Serve a frasi come «chiedi della casa» o «parla con anna del passaggio».

- **Nel core.** Una proprietà facoltativa dell'oggetto, «argomento»; la posizione «nessuno» vale solo per gli argomenti. Il controllo verifica che un argomento non sia prendibile, personaggio o contenitore. Un oggetto qualsiasi si trasforma in argomento dalla sua scheda.
- **Modello «Chiedere a un personaggio di un argomento».** Sceglie chi risponde e di che cosa (un argomento che c'è già, o uno nuovo che viene creato con il suo nome). La regola risponde se il personaggio è nella stanza; altrimenti dice una frase di riserva («Non c'è nessuno a cui chiederlo.»). Aggiunge il comando «chiedi» e le parole di collegamento se mancano. Con più personaggi nella stanza risponde quello scelto nel modello.
- **Parole di collegamento nuove.** In italiano anche «di», «del», «dello» e «della» (prima ignorate) separano due oggetti; in inglese «about». Così «parla con anna della

chiave» e «chiedi della chiave ad anna» sono la stessa frase: l'ordine non conta, perché se il primo oggetto è un personaggio e il secondo un argomento il gioco li scambia.

- **Una modifica al riconoscimento delle frasi.** Se dopo la parola di collegamento ci sono solo aggettivi, fanno parte del primo oggetto: «chiave di ferro» funziona come prima. Cambia un caso raro: «colpisci tavolo con dorata» (solo un aggettivo dopo «con») ora cerca un tavolo dorato; per il secondo oggetto serve un nome.
- **Dove valgono gli argomenti.** Solo nelle regole (e nella risposta «Non succede niente»): «guarda casa» e «prendi casa» rispondono «Qui non vedi niente del genere.», perché la casa non è un oggetto. Se un argomento ha lo stesso nome di un oggetto presente, la frase è ambigua e nessuna regola scatta: il pannello Controlli lo segnala.
- **Sistemi retro.** Un bit nuovo nei flag dell'oggetto (64 = argomento); gli interpreti ammettono gli argomenti solo nella ricerca per le regole e scambiano personaggio e argomento. Sullo Spectrum la riga digitata è lunga al massimo circa 28 caratteri, quindi «parla con giardiniere del passaggio» non ci sta: bastano «chiedi del passaggio» o «chiedi al giardiniere del passaggio».

Modello «Un personaggio che blocca un'uscita». Una guardia davanti a una porta, qualcuno che sbarra il sentiero. Si sceglie il personaggio e l'uscita (l'elenco mostra tutte le uscite del gioco), il comando che lo convince (per esempio «dai» con un oggetto, «mostra», oppure solo parlargli) e i testi. Il modello:

- aggiunge all'uscita la condizione «il personaggio si è fatto da parte» (lo stato «aperto» del personaggio) e il testo da mostrare quando il giocatore prova a passare;
- crea la regola che, quando il giocatore fa la cosa giusta, fa da parte il personaggio (e, se si vuole, gli passa l'oggetto, che sparisce dal gioco), con una risposta diversa se il giocatore non ha l'oggetto;

- dà al personaggio la descrizione «dopo» (per esempio «una guardia (si è fatta da parte)») e lo mette nella stanza dell'uscita.

Correzione (versione 46). La regola creata dal modello vale solo finché il personaggio non si è fatto da parte (condizione «chiuso»): dopo, rispondono di nuovo le sue altre regole, per esempio i dialoghi. E le regole create da un modello non vanno più sempre in fondo all'elenco: se ci sono già regole per la stessa frase (come «parla giardiniere» nell'esempio), vanno subito prima di quelle. Prima la regola nuova restava dietro a una regola che rispondeva sempre e non scattava mai, quindi il personaggio non si spostava.

Il testo «bloccata» sostituisce quello che l'uscita aveva già, e le altre condizioni dell'uscita restano: se la porta è anche chiusa a chiave, il giocatore vede la frase del personaggio. Il pannello Controlli verifica che qualche regola faccia da parte il personaggio.

Nell'esempio il giardiniere risponde su due argomenti, la casa e il passaggio («chiedi della casa», «chiedi del passaggio»).

Per dopo. I personaggi che si muovono da soli o agiscono a ogni turno (servono gli eventi a tempo, esclusi per ora perché complicano la verifica della soluzione) e i dialoghi a menu, da fare insieme alla futura modalità a scelte.

Comandi comuni (fatto)

Quattro comandi che quasi ogni avventura si aspetta, uguali sul Web, sul C64 e sullo Spectrum, in italiano e in inglese.

- **Aspetta** («aspetta», «attendi», «z»; in inglese «wait», «z»): risponde «Il tempo passa.» e non fa altro. Per ora il tempo non muove niente, perché non ci sono ancora eventi a tempo.
- **Ripeti** («ripeti», «r»; «again», «g»): rifà l'ultimo comando valido, anche una direzione. Se non c'è ancora un comando, risponde «Non succede niente.». «Ripeti» non si ripete mai da solo, quindi non può andare in giro.

- **Aiuto** («aiuto»; «help»): mostra un breve testo con qualche esempio di comando. Il testo è modificabile nella scheda Parole, tra i testi delle risposte.
- **Prendi tutto / lascia tutto** («tutto», «tutti», «tutte»; «all», «everything»): prende uno per uno gli oggetti che si possono prendere e stanno nella stanza, oppure lascia tutto quello che hai con te. Ogni riga è «etichetta: risposta», per esempio «una mela: Preso.».

Come funziona «tutto». Nel core è un oggetto speciale, un argomento con la proprietà «tutto» che non sta in nessuna stanza: serve solo perché il gioco riconosca la parola. Per questo cambia pochissimo: sul C64 e sullo Spectrum è un solo bit in più (128) nei flag dell'oggetto. Non prende gli oggetti nascosti, quelli dentro un contenitore e i personaggi.

Le regole valgono ancora. Per ogni oggetto il gioco prova prima le regole di «prendi» (o «lascia») di quell'oggetto, come se il giocatore le avesse scritte una per una. Se una regola scatta, al posto di «Preso.» compare la sua risposta; se finisce il gioco, l'elenco si ferma. Nell'esempio, «lascia tutto» in cucina con la moneta in tasca chiude la partita.

Progetti vecchi. I core già scritti non hanno questi comandi né la parola «tutto»: nella scheda Parole, sotto Comandi, compare un riquadro che dice che cosa manca, con il bottone «+ Aggiungili». I progetti nuovi li hanno già, in italiano o in inglese. Le frasi nuove («Il tempo passa.», l'aiuto, «Non c'è niente da prendere.», «Non hai niente da lasciare.») hanno un testo predefinito per lingua e si possono cambiare.

Sulle macchine retro. I comandi di base diventano nove (aspetta 7, ripeti 8, aiuto 9) e i comandi dell'autore partono da 10. «Ripeti» tiene una copia dell'ultima riga scritta (fino a 28 caratteri sullo Spectrum). Sullo Spectrum i dati del gioco cominciano ora 512 byte più avanti, per far posto al codice in più; sul C64 il codice finisce ora a $17CE, a soli 50 byte dal font (che sta a $1800): la prossima funzione grossa richiederà di spostare il font o di alleggerire l'interprete.

Stanze buie (fatto)

Una stanza può essere segnata come **buia**, e un oggetto come **fonte di luce**. Senza una luce accesa il giocatore non vede la stanza: legge solo «È buio pesto: non riesci a vedere niente.» (frase modificabile tra i testi delle risposte), non vede né può prendere gli oggetti della stanza e non ha l'elenco delle uscite, ma può comunque muoversi. Gli oggetti che ha con sé restano utilizzabili.

- **Nel core.** Due proprietà facoltative, quindi il formato resta 2: «buia» sulla stanza e «luce» sull'oggetto. L'oggetto fa luce quando è acceso (lo stato «acceso» già usato da lanterne e simili) ed è nella stanza o con il giocatore, anche solo in una tasca: dentro un contenitore chiuso non conta.
- **Nell'app.** Nella scheda Stanze la casella «Stanza buia»; nella scheda Oggetti «Fa luce quando è acceso». Per accendere la lanterna si usa il modello «Accendere e spegnere», che può anche descrivere di nuovo la stanza: così, accendendola, il giocatore vede subito dove si trova.
- **Controlli.** Segnala una stanza buia quando nessun oggetto fa luce, e una fonte di luce che nessuna regola accende.
- **Nell'esempio** il passaggio segreto è buio: si entra con la lanterna accesa, e se la si spegne lì dentro il passaggio torna buio e la moneta non si vede più.
- **Sulle macchine retro.** La stanza buia è un byte per stanza in una tabella nuova (l'intestazione dei dati passa da 30 a 32 byte), la luce è un bit in più nei flag dell'oggetto (2). L'interprete controlla la luce quando descrive la stanza, quando cerca un oggetto e per «prendi tutto». Sul C64 ho spostato il font da $1800 a $1C00 e i dati da $1C00 a $2000, per avere altro spazio per il codice (ora finisce a $1839, con poco meno di 1 KB libero); sullo Spectrum il codice finisce a $90B6 e i dati cominciano a $9200.

La guida nell'app e l'app in inglese (versione 50)

Guida. C'è una scheda nuova, **Guida**, con la guida divisa in 13 sezioni che si aprono e si chiudono, e un indice in cima per saltare all'argomento. Il bottone **?** in alto apre direttamente la

sezione della scheda in cui ci si trova (dalla scheda Oggetti, la sezione Oggetti). Il testo della guida è nell'app in italiano e in inglese.

Lingua dell'app. Il bottone **IT/EN** in alto cambia la lingua dell'interfaccia: schede, bottoni, spiegazioni, controlli, modelli di regola, guida. La scelta viene ricordata; per applicarla la pagina si ricarica (il progetto è già salvato). È indipendente dalla lingua del gioco: con l'app in inglese si può scrivere un'avventura in italiano, e viceversa. Tecnicamente i testi dell'interfaccia restano scritti in italiano nel codice e passano da una funzione T(), che li cerca in un dizionario inglese di circa 800 voci incluso nella pagina; i testi del gioco (core, risposte predefinite, esempio) non passano da lì.

Pubblicazione su GitHub Pages (fatto)

L'app è nel repository pubblico **Linobob/avventure** (licenza MIT): il file `index.html` è l'app intera, in una sola pagina, e GitHub Pages la serve all'indirizzo linobob.github.io/avventure/. Il repository contiene anche i giochi d'esempio compilati (cartella esempi), i documenti in PDF (documenti) e tutti i sorgenti (sorgenti): compilatori, interpreti in assembly, assemblatori ed emulatori di prova, test.

- **Una sola fonte.** La pagina pubblicata si prepara con lo script `pubblica.py`, che rigenera il motore, copia l'app in `index.html`, ricompila gli esempi (C64, Spectrum, Web, italiano e inglese) e crea l'elenco `NOVITA.md`.
- **Pagina adatta ai telefoni.** Dentro Claude la pagina riceve da sola la larghezza del telefono; fuori no. Lo script la avvolge in un documento completo, con il tag «viewport», così sul telefono non appare più come un sito da computer rimpicciolito.
- **Link per tornare al sito.** In alto a sinistra c'è «← Linobob», come nelle altre pagine del sito personale. Compare solo nella versione pubblicata.
- **Elenco delle novità.** Il bottone ✦ in alto apre l'elenco delle novità del motore (nuove caratteristiche utili a chi scrive

avventure), in italiano e in inglese. Non contiene le modifiche tecniche o del sito. Si aggiorna a mano, in un blocco dell'app che lo script riporta anche in `NOVITA.md`.

Salvare i file su iPhone e iPad (fatto)

Su iPhone e iPad (da iOS 18.2) i download che partono da un indirizzo «blob», cioè un file creato dalla pagina stessa, spesso non arrivano nell'app File: è un difetto di Safari e di altri browser, segnalato in molte discussioni. L'app lo aggira usando il **pannello Condividi** di iOS: quando si salva un file (core .json, gioco .html, .prg, .tap) compare un riquadro con «Salva» e «Annulla»; toccando «Salva» si apre il pannello e si sceglie «Salva su File». Il riquadro serve perché iOS apre il pannello solo dopo un tocco diretto. Sugli altri dispositivi il salvataggio non cambia.

Combinare due oggetti (fatto)

Nuovo modello di regola **«Combinare due oggetti»**: «combina corda con gancio» fa sparire i due oggetti e ne crea uno nuovo, che va con il giocatore. Non serve nessun motore nuovo: è una regola a due oggetti, con tre azioni (sposta il primo fuori dal gioco, sposta il secondo fuori dal gioco, sposta il risultato con il giocatore) e il messaggio.

- **Che cosa chiede.** Il comando (se manca, aggiunge «combina» con i sinonimi unisci e assembla), i due oggetti, che cosa ne nasce (un oggetto nuovo, con nome, riga dell'elenco e descrizione, oppure uno già preparato fuori dal gioco), il messaggio, se il giocatore deve avere con sé tutti e due e il messaggio per quando non li ha.
- **I due ordini.** Una regola vale per un ordine solo («combina A con B»), quindi il modello può crearne due, anche per «combina B con A» (spunta attiva di default).
- **Oggetti fuori dal gioco.** Per far nascere l'oggetto serve un posto dove stia prima: nella scheda di un oggetto, «Dove si trova all'inizio» ha ora la voce **«Fuori dal gioco (una regola lo fa comparire)»**. Prima valeva solo per gli argomenti di conversazione. Il formato del core non cambia (il valore

«nessuno» era già previsto) e i compilatori C64 e Spectrum lo trattavano già come «non c'è». Il pannello Controlli avvisa se un oggetto parte fuori dal gioco e nessuna regola lo fa comparire.

- **Prove.** Sul gioco d'esempio, combinando chiave e moneta compare l'amuleto con entrambi gli ordini; Web, C64 e Spectrum danno le stesse risposte, e l'esplorazione completa del gioco resta di 30.024 situazioni.

Spostare regole, eventi e passi (versione 47)

Nelle schede Regole, Eventi e Soluzione, oltre ai bottoni «Su» e «Giù» (un posto alla volta), ogni elemento ha un menu **«Sposta in…»**: elenca tutte le posizioni possibili («In cima», «Prima della 5: parla giardiniere», «In fondo») e sposta l'elemento subito, in un solo passaggio. Dopo lo spostamento la scheda resta aperta e la pagina scorre fino a lei. L'ordine delle regole conta, perché scatta la prima che va bene: per questo il menu mostra il riassunto della regola davanti alla quale ci si mette.

Modelli di regola (fatto)

Nella scheda Regole c'è un riquadro **«Nuova regola da modello»**: scegli il tipo di situazione, compili pochi campi e l'app crea le regole complete. Il risultato è fatto di regole normali, quindi poi si modificano, si spostano e si eliminano come tutte le altre. Prima di creare, l'anteprima elenca cosa verrà aggiunto e dice se manca qualcosa (un secondo oggetto, una parola di collegamento come «con», un testo).

- **Aprire o sbloccare con un altro oggetto** (porta e chiave): 3 regole, cioè già aperto, aperto con l'oggetto giusto (con la risposta se non lo si ha con sé) e la risposta quando si prova senza niente.
- **Accendere e spegnere**: con un solo comando (per esempio «usa») le regole alternano; con due comandi diversi ce ne sono quattro, con «è già acceso» e «è già spento». Si può far descrivere di nuovo la stanza dopo il cambio, utile per le luci.

- **Mangiare, bere o consumare**: l'oggetto sparisce dal gioco; deve essere in mano al giocatore o basta che sia nella stanza.
- **Un oggetto che non si può prendere**, con una risposta pensata per quell'oggetto al posto di quella generica.
- **Usare un oggetto**, il modello più flessibile: un comando su un oggetto, facoltativamente con un secondo oggetto e solo in una stanza, con la richiesta che il giocatore li abbia con sé e un risultato a scelta (solo una risposta, aprire o accendere un oggetto, spostarne uno, far sparire quello usato, far finire il gioco) più la risposta se le condizioni non sono vere.
- **Il gioco finisce quando…**: il giocatore lascia un oggetto in una stanza (una regola) oppure ci entra con l'oggetto (un evento d'ingresso).
- **Una risposta particolare a un comando**, con uno o due oggetti.

Dettagli. I campi partono da valori sensati (per esempio la porta è l'oggetto che ha già una variante «aperto»; la lanterna quello che ha una variante «acceso»; i testi predefiniti sono in italiano o in inglese secondo la lingua base). Le regole nuove vanno in fondo all'elenco: se una regola precedente risponde sempre alla stessa frase, la nuova non scatta mai e la scheda Controlli lo segnala. Un modello non conosce lo stato del gioco oltre a quello che serve a lui: per situazioni più complesse si usa l'editor normale, partendo da un modello e completando a mano.

Pannello Controlli (fatto)

La scheda **Controlli** raccoglie in un posto solo tutti i controlli sul core, che prima erano sparsi tra Mappa, Soluzione e Compila. Ogni voce ha una gravità (errore, avviso, nota), una categoria e un pulsante «Vai» che apre la scheda e la voce da correggere. Se il core non è valido si vedono solo gli errori; altrimenti gli avvisi sono ordinati per categoria.

- **Stanze:** stanze che non si raggiungono dall'inizio, stanze senza uscite dove il giocatore resterebbe bloccato, descrizioni mancanti, stanze più lunghe di una schermata sui

sistemi retro (circa 650 caratteri tra descrizione e frammenti), eventi d'ingresso legati a stanze irraggiungibili.

- **Oggetti:** oggetti che non si trovano mai (in una stanza irraggiungibile e senza regole che li spostano), nomi che coincidono con una parola ignorata, due oggetti con lo stesso nome senza un aggettivo che li distingua, oggetti nascosti e mai nominati nel testo, descrizioni mancanti.
- **Regole:** una regola che non scatta mai perché una precedente risponde sempre alla stessa frase, regole con «con» ma senza parole di collegamento nel vocabolario, comandi aggiunti senza nessuna regola, condizioni su uno stato che nessuna regola produce (un oggetto che deve essere aperto o acceso quando nessuna azione lo apre o lo accende).
- **Parole:** la stessa parola con più significati (due comandi, un comando e un nome di oggetto, ecc.).
- **Variabili:** variabili mai usate o mai controllate, e condizioni su una variabile che nessuna azione cambia e che non possono mai essere vere con il suo valore iniziale.
- **Finale e soluzione:** nessuna regola fa finire il gioco, soluzione vuota o senza il passo finale. La verifica vera e propria (esplorazione, vicoli ciechi) resta nella scheda Soluzione.
- **Testi e lingue:** caratteri che sui sistemi retro diventerebbero «?» (controllati per ogni lingua, saltando i testi non ancora tradotti) e traduzioni incomplete.
- **Memoria:** per ogni lingua, quanto spazio occupano il gioco per C64 e per Spectrum, con una barra rispetto alla memoria disponibile (circa 38 KB sul C64 e 28 KB sullo Spectrum), più gli avvisi dei compilatori, come le parole troppo lunghe.

Cosa non fa ancora. I controlli sono statici: non simulano la partita. Una regola o un oggetto raggiungibili solo con una sequenza particolare di mosse possono risultare «a posto» pur non funzionando; per questo la scheda Soluzione continua a servire. La barra di stato in alto mostra ancora solo il primo

problema.

Inglese e traduzioni (fatto)

La proposta del capitolo «Più lingue in un solo core» è diventata la scheda **Lingue**. Un core ha una **lingua base** (italiano o inglese) e, se serve, una o più **traduzioni**; ogni gioco compilato ha una lingua sola, scelta nelle schede Prova e Compila. Il gioco di esempio contiene già la sua traduzione inglese, «The Empty House».

- **Nel core.** Due campi facoltativi, quindi il formato resta 2: «lingua» (assente = italiano) e «traduzioni», per esempio traduzioni.en = { testi, vocabolario, oggetti }. Ogni testo tradotto ha una chiave stabile (per esempio «stanza.cucina.descrizione» o «regola.3.altrimenti») e ricorda l'originale da cui è stato tradotto. Se l'originale cambia, la traduzione è «da rivedere»; se una regola si sposta, la traduzione si ritrova lo stesso perché si riconosce dal testo originale.
- **Testi.** Titolo, introduzione, nomi e descrizioni delle stanze, frammenti, testi delle uscite, etichette e descrizioni degli oggetti (anche aperti e accesi), risposte e finali delle regole e degli eventi, risposte del gioco. Le risposte del gioco che l'autore non ha cambiato non vanno tradotte: ogni lingua ha le sue frasi predefinite.
- **Vocabolario per lingua.** Comandi e direzioni si traducono per identificativo (il comando «prendi» in inglese è take, get, pick, grab); poi preposizioni, parole ignorate e, per ogni oggetto, nome, sinonimi e aggettivi. Il programma suggerisce le parole comuni (una tabella italiano-inglese di circa trenta comandi e dodici direzioni). In una traduzione gli identificativi non sono parole del gioco: contano solo le parole elencate.
- **Parser inglese.** Lo stesso motore, con una regola nuova valida per tutte le lingue: una preposizione che viene prima di qualsiasi nome non separa i due oggetti, così «turn on lamp» vale come «turn lamp on» e «look at door» come «look door». I verbi con particella funzionano scrivendo solo il verbo: in «pick up lantern» la parola «up» è una direzione e il

gioco la salta. Fatto in JavaScript e negli interpreti C64 e Spectrum.

- **Cosa cambia nel gioco compilato.** Testi e parole della lingua scelta, le risposte predefinite della lingua, la congiunzione degli elenchi («a key and a table»), la parola per ricominciare («restart») e, nel gioco Web, le scritte della pagina e la lingua del documento.
- **Traduzione assistita.** «Esporta» crea un file JSON con le istruzioni per il modello (lingue, tono, segnaposto, una sola parola per voce del vocabolario, lettere disponibili sui sistemi retro) e l'elenco di testi e parole con chiave, contesto e originale. Si può esportare solo ciò che manca o è da rivedere. La risposta si incolla nella scheda o si apre come file: il programma la controlla (chiavi che non esistono più, segnaposto, parole con spazi), mostra cosa cambierà e importa solo dopo conferma.
- **Progetti in inglese.** Nella scheda Core, «Nuovo progetto vuoto in inglese» prepara un core con lingua base inglese, risposte e vocabolario inglesi.
- **Limiti.** Per ora le lingue conosciute sono italiano e inglese. Nell'app l'interfaccia resta in italiano. Gli enigmi che dipendono dalla lingua non hanno ancora regole valide solo per certe lingue.

Accenti veri sui sistemi retro (fatto)

Prima i giochi per C64 e Spectrum scrivevano le lettere accentate con l'apostrofo (a', e'). Ora si vedono le lettere vere: à è é ì ò ù e le maiuscole À È É Ì Ò Ù (anche quelle con accento acuto o grave scambiate, come á ó ú, sono rese con la forma più vicina).

- **Commodore 64.** Il font del gioco passa da 96 a 128 caratteri: i codici schermo 96-107 sono le dodici lettere accentate, disegnate in stile con le altre (accento nelle righe 0-1 sopra le minuscole, nella sola riga 0 sopra le maiuscole). Il programma ora occupa fino a $1A00 per font e dati: il font sta a $1600-$19FF, i dati del gioco iniziano a $1A00.

- **ZX Spectrum.** Le dodici lettere sono caratteri definiti dall'utente (codici 144-155) in una piccola tabella dentro l'interprete; all'avvio il gioco punta la variabile di sistema UDG a quella tabella. Il disegno delle lettere di base segue il font della ROM.
- **Cosa non cambia.** I comandi del giocatore restano senza accenti (sulle tastiere originali non si scrivono) e le parole del vocabolario si confrontano senza accenti. Gli altri caratteri non ASCII continuano a diventare «?» sui sistemi retro.

Compatibilità dei core nel tempo (proposta)

Obiettivo: un core scritto con una versione dell'app deve poterlo aprire e compilare anche quelle successive, senza perdere nulla.

- **Il core porta un numero di formato** (oggi 2). Le novità che si possono aggiungere senza cambiare la forma dei dati (una nuova proprietà facoltativa di una stanza, un nuovo tipo di azione, un nuovo comando di base) **non cambiano il numero**: i core vecchi non hanno quel campo e il motore, in sua assenza, si comporta come prima.
- **Quando la forma dei dati deve cambiare davvero** (per esempio un testo che diventa «un testo per lingua»), il numero sale e l'app contiene una **funzione di migrazione** dal formato N al N+1. All'apertura di un core vecchio, dal browser o da un file, l'app lo converte in automatico, avvisa e conserva una copia dell'originale. Le migrazioni si concatenano ($2 \to 3 \to 4$), quindi anche un core molto vecchio si aggiorna.
- **Controllo oggi:** il formato deve valere esattamente 2 e non esiste ancora nessuna migrazione, perché c'è una sola versione. Il meccanismo (un elenco di migrazioni e il rifiuto di un core più nuovo dell'app, con messaggio chiaro) si prepara la prima volta che serve.

- **Test di non regressione:** i core di esempio di ogni versione restano nel progetto; a ogni modifica si controlla che si aprano, si convertano, si compilino e si portino a termine su tutte le piattaforme.
- **I giochi già compilati non dipendono dall'app:** ogni gioco contiene già il suo motore (Web) o il suo interprete (C64 e Spectrum). Un aggiornamento non rompe i file già distribuiti; per averne i miglioramenti si ricompila il core.
- **Copie di sicurezza:** il core è nella memoria del browser, che può essere cancellata. Conviene salvarlo ogni tanto in un file (scheda Core, «Salva in un file»); più avanti si può proporre un promemoria o una copia automatica.

Più lingue in un solo core (proposta)

Scelta proposta: un solo core per tutte le lingue, non un core per lingua. Con un core per lingua, ogni modifica a un enigma, a una porta o a una regola va rifatta in ogni copia, e le copie col tempo smettono di coincidere. Con un core solo, la struttura (stanze, uscite, oggetti, regole, soluzione) è una e le lingue cambiano solo le parole.

Cosa dipende dalla lingua e cosa no. Non dipendono: gli identificativi, le uscite, le posizioni, le condizioni, le azioni, la soluzione. Dipendono: nomi e descrizioni di stanze e oggetti, etichette, testi delle regole e dei finali, le frasi di risposta del gioco, le note della soluzione e il **vocabolario** (comandi, direzioni, parole ignorate e di collegamento, nomi e sinonimi degli oggetti). Il vocabolario è un caso a parte, perché non è una traduzione parola per parola: dipende dal parser della lingua (l'inglese ha verbi con particella come «pick up» e «look at»; l'italiano ha forme come «prendilo»), e il parser per ogni lingua resta una decisione già presa.

Come si memorizza. Il core ha una **lingua base**, quella in cui si scrive (oggi l'italiano), con i testi dove sono ora. Le altre lingue stanno in una sezione a parte, **«traduzioni»**, che associa a ogni testo una chiave stabile (per esempio la descrizione della stanza «cucina», l'etichetta dell'oggetto «chiave», il testo della prima azione della regola «porta») il testo tradotto. Così un core

esistente non cambia: la sezione è facoltativa. Per avere chiavi stabili le regole e le azioni avranno un identificativo, che oggi non hanno. Il vocabolario di ogni lingua è una sezione propria, con un modello pronto (per l'inglese un vocabolario di base e le frasi predefinite).

Controllo delle traduzioni. Ogni testo tradotto ricorda da quale testo originale è stato tradotto: se l'originale cambia, la traduzione risulta **«da rivedere»**. Ogni lingua ha uno stato per voce (da tradurre, tradotto, da rivedere) e un conteggio di completezza; l'app avvisa se si compila una lingua incompleta. Controlla anche che i segnaposto come «{0}» siano rimasti, e che i testi non superino i limiti dei sistemi retro e usino solo caratteri disponibili.

Come si traduce. Due modi, con gli stessi dati:

- **Pagina «Traduzioni» nell'app:** scegli la lingua; vedi i testi raggruppati per stanza, oggetto, regola, messaggi e vocabolario, con l'originale accanto al campo da compilare, il suo contesto (per esempio «risposta a: apri porta con chiave») e lo stato.
- **File per una traduzione assistita:** l'app esporta tutti i testi da tradurre in un file semplice (chiave, originale, contesto, eventuale lunghezza massima, e la traduzione da compilare), con in testa le istruzioni (lingua di arrivo, tono, segnaposto da conservare, nomi da non cambiare). Lo si dà a un modello come Claude e si importa il risultato, che l'app controlla prima di accettarlo e mostra per intero per la revisione. Si può esportare solo ciò che manca o che è «da rivedere», così dopo una modifica non si ritraduce tutto.

Compilazione. Resta una lingua per gioco compilato, come già deciso, senza scelta a runtime: dallo stesso core si compila un gioco per lingua e piattaforma. Nel Web si potrebbe includere più lingue in un solo file, ma sui sistemi retro la memoria lo impedisce, quindi conviene lo stesso comportamento ovunque.

Casi particolari. Gli enigmi che dipendono dalla lingua (giochi di parole, anagrammi, parole d'ordine) non si traducono: per questi una regola può essere valida solo per certe lingue. Le lingue con

lettere diverse da quelle dell'inglese (francese, spagnolo, tedesco) richiedono le lettere accentate vere sui sistemi retro, già nell'elenco della prima versione.

Scorrimento del testo sui sistemi retro (fatto)

Prima il testo nuovo faceva salire lo schermo di una riga per ogni riga stampata: una risposta di sei righe dava sei scatti di fila, fastidiosi da seguire, e una risposta più lunga dello schermo scappava in alto prima di poterla leggere. Ora ci sono due modi, scelti nella scheda Compila (riquadro «Opzioni per C64 e Spectrum») e salvati nel progetto (campo facoltativo `opzioni_retro` del core):

- **A blocchi** (predefinito): il gioco prepara tutta la risposta in un'area di memoria, conta le righe e fa scorrere lo schermo una volta sola, di quanto serve, poi la stampa. Un solo scatto per comando.
- **Una riga alla volta:** come prima, lo scorrimento lo fa la ROM a ogni riga.
- **Pausa a schermo pieno**, in tutti e due i modi: se una risposta non sta nello schermo (24 righe sul C64, 21 sullo Spectrum, oltre alla riga del comando), il gioco scrive «[premi un tasto]», aspetta, cancella la frase e continua con la pagina seguente. Ogni riga si vede almeno una volta.
- **Attesa dopo l'introduzione** (spunta, attiva di default): dopo il titolo e l'introduzione il gioco aspetta un tasto prima di mostrare la prima stanza.
- La frase della pausa è un testo delle risposte nuovo, «premi_tasto» («[premi un tasto]», «[press a key]»), modificabile e traducibile come gli altri; sta su una riga sola (al massimo 30 caratteri).

Come funziona. Tutto il testo passa da una routine sola, `putc`. Nel modo a blocchi i caratteri (con i codici di colore) vanno in un buffer di 1,25 KB ($C300 sul C64, $F500 sullo Spectrum) e vengono contate le righe. Prima di leggere il comando successivo, `flushb` calcola quante righe mancano in fondo allo schermo, fa scorrere tutto in un colpo (sul C64 schermo e colori,

poi rimette in ordine la tabella delle righe del KERNAL; sullo Spectrum pixel e attributi delle 22 righe in alto, riga di testo per riga di testo), sposta il cursore e stampa il buffer. Le opzioni stanno in un byte libero dell'intestazione dei dati (bit 0 a blocchi, bit 1 attesa dopo l'introduzione), quindi il codice è lo stesso per tutti i giochi.

Memoria. Il codice nuovo occupa circa 740 byte sul C64 e 510 sullo Spectrum. Sul C64 resta sotto il font ($1C00), ma con soli 230 byte di margine: le prossime funzioni del C64 dovranno spostare il font e i dati più in alto. Sullo Spectrum non c'era abbastanza spazio tra il codice e i dati, quindi i dati partono da $9400 invece che da $9200: restano 512 byte in meno per i dati del gioco.

Più avanti: lo scorrimento morbido sul C64 (lo scroll fine dell'hardware, pixel per pixel, con un interrupt di raster), una zona fissa in alto per il nome della stanza e poi per le immagini, e le opzioni cambiabili dal giocatore direttamente nel gioco.

Immagini e musica (da fare più avanti)

Da aggiungere dopo il supporto per le avventure grandi (.d64, testi caricati dal disco e compressi), perché immagini e musica occupano molta più memoria dei testi.

Immagini. Ogni stanza può avere un'immagine (più avanti anche oggetti ed eventi, per esempio una scena). Nel core c'è solo il riferimento all'immagine; i file stanno nel progetto, insieme al core.

- **Web:** l'immagine originale, ridimensionata, sopra il testo.
- **Commodore 64:** bitmap multicolore (160×200, 4 colori per blocco di 4×8 pixel, uno in comune) o ad alta risoluzione (320×200, 2 colori per blocco di 8×8). L'immagine occupa la parte alta dello schermo e il testo scorre sotto: con un interrupt di raster lo schermo è diviso tra bitmap in alto e testo in basso.

- **ZX Spectrum:** 256×192 con 2 colori per blocco di 8×8 (più la luminosità). Lo schermo è già tutto bitmap, quindi immagine e testo convivono senza trucchi; il limite è il colore per blocco («attribute clash»).
- **Memoria:** un'immagine a metà schermo costa circa 4-5 KB sul C64 e circa 3,5 KB sullo Spectrum, prima della compressione. Sul C64 le immagini si caricano dal disco quando servono; sullo Spectrum 48K su nastro ne entrano poche, e per molte immagini servono il 128K o un caricamento a più parti.

Convertitore automatico nell'app. Nella scheda della stanza si carica una foto o un disegno e l'app lo converte per ogni sistema, nel browser: ritaglia e ridimensiona, riduce i colori alla tavolozza della macchina, applica il dithering (a scelta: nessuno, ordinato, a diffusione) e rispetta i limiti di colore per blocco. Un'anteprima mostra l'immagine come apparirà davvero su C64 e Spectrum, accanto all'originale, e il pannello della memoria dice quanto spazio occupa. Più avanti si potrà ritoccare a mano il risultato.

Musica. Una musica per il titolo e per il finale, e magari per stanze o eventi. Convertire in automatico un file musicale qualsiasi non è realistico, quindi:

- **Web:** un file audio, oppure la stessa musica dei sistemi retro riprodotta nel browser.
- **Commodore 64:** il chip SID. Due strade: importare una musica già pronta nel formato .sid (con il suo player), oppure scriverla in un piccolo editor di note nell'app, che il compilatore traduce per il SID.
- **ZX Spectrum:** il 48K ha solo il beeper, che blocca il programma mentre suona: va bene per titolo e finale, non durante il gioco. Lo Spectrum 128K ha il chip AY, a tre voci, che suona in sottofondo.
- La musica dell'editor di note si compila per tutti i sistemi, così la si scrive una volta sola.

Questioni aperte

Decisioni prese:

- Gli interpreti per C64 e Spectrum sono in linguaggio macchina (6502 e Z80).
- Un parser per ogni lingua, scelto in compilazione; italiano e inglese dall'inizio.
- Un solo core per tutte le lingue, con una lingua base e le traduzioni in una sezione a parte (proposta, da confermare).
- La modalità a scelte multiple è un'estensione successiva.
- Il core è in JSON; un linguaggio leggibile, scrivibile a mano, si genera e si importa a richiesta.
- L'editor delle regole è visuale, con la possibilità di intervenire sullo script (la vista testuale della regola).
- Gli eventi casuali o a tempo restano fuori dalla soluzione per ora.
- La compilazione avviene nel browser, dentro l'app, se tecnicamente possibile.
- Lettere accentate: all'inizio vocale più apostrofo (e', a'), poi si rivede.
- L'app sarà pubblica, con licenza MIT per tutto (app, compilatori, interpreti) per ora; i giochi creati restano dell'autore.
- Altri target futuri: altri home computer dell'epoca e MS-DOS, poi si vedrà.

Nessuna questione aperta per ora. Da riprendere più avanti: i salvataggi e il caricamento dei testi da disco o cartuccia (vedi la proposta sopra), la compressione dei testi sui sistemi retro, gli eventi casuali o a tempo nella soluzione, una soluzione migliore per le lettere accentate, e la verifica delle licenze degli strumenti usati prima di pubblicare.

Rischi principali: la quantità di testo che entra in memoria sui sistemi retro, e la coerenza del parser tra interpreti scritti in linguaggi diversi. Il secondo si riduce con una specifica unica e una batteria di test comuni, da far girare su ogni interprete.